TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN: CÔNG NGHỆ PHẦN MỀM

ĐỀ THI VÀ BÀI LÀM

Tên học phần: Toán ứng dụng CNTT
Mã học phần: Hình thức thi: *Tự luận có giám sát*
Đề số: **Đ02** Thời gian làm bài: 90 phút *(không kể thời gian chép/phát đề)*
Được sử dụng tài liệu khi làm bài.

Họ tên: Huỳnh Mỹ Ngọc…**Lớp**:…24T_DT4……**MSSV**:…102240207……...

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MS Teams.

Câu 1 (*2.0 điểm, đáp ứng CLO1,CLO2, CLO4*)

Viết chương trình (có sử dụng hàm) thực hiện công việc sau, biết rằng n=50:

a) *(1.0 điểm)* Cho dãy Fibonacci:

$$F(1) = F(2) = 1;\ F(n) = F(n-1) + F(n-2),\ (n > 2).$$

Tìm tất cả các số Fibonacci trong n số Fibonacci đầu tiên có đúng 6 chữ số (tức nằm trong khoảng [100000, 999999]). Liệt kê các số tìm được và tính tổng của chúng.

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <vector>

using namespace std;

vector<long long> generateFibonacci(int n) {
    vector<long long> fib(n + 1);

    fib[1] = 1;
    fib[2] = 1;

    for (int i = 3; i <= n; i++) {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    return fib;
}

void findSixDigitFibonacci(const vector<long long>& fib, int n) {
    long long sum = 0;

    cout << "Cac so Fibonacci co dung 6 chu so trong "
         << n << " so Fibonacci dau tien la:\n";

    for (int i = 1; i <= n; i++) {
        if (fib[i] >= 100000 && fib[i] <= 999999) {
            cout << fib[i] << " ";
            sum += fib[i];
        }
    }

    cout << "\nTong cac so Fibonacci co dung 6 chu so la: " << sum << endl;
}

int main() {
    int n = 50;

    vector<long long> fib = generateFibonacci(n);

    findSixDigitFibonacci(fib, n);

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Cac so Fibonacci co dung 6 chu so trong 50 so Fibonacci dau tien la:
121393 196418 317811 514229 832040
Tong cac so Fibonacci co dung 6 chu so la: 1981891
```

b) (1.0 điểm) Trong n số Fibonacci đầu tiên, hãy liệt kê các số Fibonacci chia hết cho 3 và tính tổng các số này

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
#include <vector>

using namespace std;

// Tao n so Fibonacci dau tien
vector<long long> generateFibonacci(int n) {
    vector<long long> fib(n + 1);

    fib[1] = 1;
    fib[2] = 1;

    for (int i = 3; i <= n; i++) {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    return fib;
}

// Liet ke cac so Fibonacci chia het cho 3 va tinh tong
void findFibonacciDivisibleBy3(const vector<long long>& fib, int n) {
    long long sum = 0;

    cout << "Cac so Fibonacci chia het cho 3 trong "
         << n << " so Fibonacci dau tien la:\n";

    for (int i = 1; i <= n; i++) {
        if (fib[i] % 3 == 0) {
            cout << fib[i] << " ";
            sum += fib[i];
        }
    }

    cout << "\nTong cac so Fibonacci chia het cho 3 la: " << sum << endl;
}

int main() {
    int n = 50;

    vector<long long> fib = generateFibonacci(n);
```

```
    findFibonacciDivisibleBy3(fib, n);

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Cac so Fibonacci chia het cho 3 trong 50 so Fibonacci dau tien la:
3 21 144 987 6765 46368 317811 2178309 14930352 102334155 701408733 4807526976
Tong cac so Fibonacci chia het cho 3 la: 5628750624
```

Câu 2 (*1.5 điểm, đáp ứng CLO1,CLO2, CLO3, CL04*)

Cho ma trận A. Viết chương trình (có sử dụng hàm) thực hiện phân rã ma trận A bằng phương pháp SVD.

Trả lời: Dán code vào bên dưới

```
#include <bits/stdc++.h>
#include <iomanip>
using namespace std;
#include <Eigen/Dense>

class Matrix {
  int row, col;
  double **data = nullptr;
public:
  Matrix(int row = 0, int col = 0);
  Matrix(const Matrix &other);
  ~Matrix();
  Matrix Transpose();
  bool IsSquare();
  double Det();
  int Row() const;
  int Col() const;
  Matrix GetColumn(int k);
  Matrix &operator=(const Matrix &other);
  double &operator()(int row, int col);
  double operator()(int row, int col) const;

  operator Eigen::MatrixXd() const;

  friend ostream &operator<<(ostream &os, const Matrix &m);
  friend istream &operator>>(istream &is, Matrix &m);
  friend Matrix operator*(const Matrix &a, const Matrix &b);

  friend Matrix operator/(const Matrix &m, double k);
};
```

```
void Calculate_Sigma_and_V(Eigen::SelfAdjointEigenSolver<Eigen::MatrixXd> &solver,
Matrix &Sigma, Matrix &V) {
  int m = V.Row();

  vector<pair<double, int>> eigen_vals;
  for (int i = 0; i < m; ++i) {
    eigen_vals.push_back({solver.eigenvalues()(i), i});
  }

  sort(eigen_vals.begin(), eigen_vals.end(), [](const pair<double, int>& a, const pair<double,
int>& b) {
    return a.first > b.first;
  });

  for (int i = 0; i < m; ++i) {
    for (int j = 0; j < Sigma.Row(); ++j) {
      Sigma(j, i) = 0;
    }

    double val = eigen_vals[i].first;
    Sigma(i, i) = (val > 1e-5) ? sqrt(val) : 0.0;

    int original_idx = eigen_vals[i].second;
    for (int j = 0; j < m; ++j) {
      V(j, i) = solver.eigenvectors()(j, original_idx);
    }
  }
}

void Calculate_U(Matrix &Sigma, Matrix &A, Matrix &V, Matrix &U) {
  int n = U.Row();
  int m = V.Col();
  int count = 0;

  for(int i = 0; i < m; ++i) {
    if(Sigma(i, i) < 1e-5) continue;

    Matrix Vi = V.GetColumn(i);
    Matrix Ui = A * Vi;
    Ui = Ui / Sigma(i, i);

    for(int j = 0; j < n; ++j) {
      U(j, count) = Ui(j, 0);
    }
    count++;
  }

  for (int i = 0; i < n && count < n; ++i) {
    Matrix v(n, 1);
```

```
      for (int j = 0; j < n; ++j) {
         v(j, 0) = (i == j) ? 1.0 : 0.0;
      }

      for (int j = 0; j < count; ++j) {
         double dot_product = 0;
         for (int k = 0; k < n; ++k) {
            dot_product += U(k, j) * v(k, 0);
         }
         for (int k = 0; k < n; ++k) {
            v(k, 0) -= dot_product * U(k, j);
         }
      }

      double norm = 0;
      for (int k = 0; k < n; ++k) {
         norm += v(k, 0) * v(k, 0);
      }
      norm = sqrt(norm);
      if (norm > 1e-5) {
         for (int k = 0; k < n; ++k) {
            U(k, count) = v(k, 0) / norm;
         }
         count++;
      }
   }
}

void PrintAnswer(Matrix &U, Matrix &Sigma, Matrix &V, bool transpose) {
   if(!transpose) {
      cout << "U: " << endl << U << endl;
      cout << "Sigma: " << endl << Sigma << endl;
      cout << "VT: " << endl << V.Transpose() << endl;
      cout << "U * Sigma * VT: " << endl << U * Sigma * V.Transpose() << endl;
   }
   else {
      cout << "U: " << endl << V << endl;
      cout << "Sigma: " << endl << Sigma.Transpose() << endl;
      cout << "VT: " << endl << U.Transpose() << endl;
      cout << "U * Sigma * VT: " << endl << V * Sigma.Transpose() * U.Transpose() << endl;
   }
}

int main() {
   bool isTranspose = false;
   int n, m;
   cout << "Nhap kich thuoc ma tran: ";
   cin >> n >> m;
   Matrix A(n, m);
```

```
  cout << "Nhap ma tran A: " << endl;
  cin >> A;

  if(n < m) {
     isTranspose = true;
     swap(n, m);
     A = A.Transpose();
  }

  Matrix U(n, n), Sigma(n, m), V(m, m);
  Matrix S = A.Transpose() * A;

  Eigen::SelfAdjointEigenSolver<Eigen::MatrixXd> solver((Eigen::MatrixXd)S);

  Calculate_Sigma_and_V(solver, Sigma, V);
  Calculate_U(Sigma, A, V, U);
  PrintAnswer(U, Sigma, V, isTranspose);

  return 0;
}

Matrix::Matrix(int row, int col) : row(row), col(col) {
  data = new double*[row];
  for(int i = 0; i < row; ++i) {
     data[i] = new double[col];
     for (int j = 0; j < col; ++j) data[i][j] = 0;
  }
}

Matrix::Matrix(const Matrix &other) : row(other.row), col(other.col) {
  data = new double*[row];
  for(int i = 0; i < row; ++i) {
     data[i] = new double[col];
     for(int j = 0; j < col; ++j)
        data[i][j] = other.data[i][j];
  }
}

Matrix::~Matrix() {
  for(int i = 0; i < row; ++i) {
     delete[] data[i];
  }
  delete[] data;
}

Matrix Matrix::Transpose() {
  Matrix T(col, row);
  for(int i = 0; i < col; ++i) {
     for(int j = 0; j < row; ++j) {
```

```
            T.data[i][j] = data[j][i];
        }
    }
    return T;
}

bool Matrix::IsSquare() {
    return row == col;
}

double Matrix::Det() {
    if(!IsSquare()) throw invalid_argument("Ma tran tinh dinh thuc phai la ma tran vuong");
    double res = 1;
    for(int i = 0; i < row; ++i) {
        if(data[i][i] == 0) {
            for(int j = i + 1; j < row; ++j) {
                if(data[j][i] != 0) {
                    for(int k = 0; k < col; ++k) {
                        double t = data[i][k];
                        data[i][k] = data[j][k];
                        data[j][k] = t;
                    }
                }
            }
        }
        if(data[i][i] == 0) return 0;
        res = res * data[i][i];
        for(int j = i + 1; j < row; ++j) {
            double ratio = -data[j][i]/data[i][i];
            for(int k = i; k < col; ++k) {
                data[j][k] = data[i][k] * ratio + data[j][k];
            }
        }
    }
    return res;
}

int Matrix::Row() const { return row; }
int Matrix::Col() const { return col; }

Matrix Matrix::GetColumn(int k) {
    Matrix res(row, 1);
    for(int i = 0; i < row; ++i) {
        res(i, 0) = data[i][k];
    }
    return res;
}

Matrix &Matrix::operator=(const Matrix& other) {
```

```
  if (this == &other) return *this;

  for(int i = 0; i < row; ++i) {
    delete[] data[i];
  }
  delete[] data;

  row = other.row;
  col = other.col;
  data = new double*[row];
  for(int i = 0; i < row; ++i) {
    data[i] = new double[col];
    for(int j = 0; j < col; ++j) {
      data[i][j] = other.data[i][j];
    }
  }
  return *this;
}

double &Matrix::operator()(int row, int col) {
  return data[row][col];
}

double Matrix::operator()(int row, int col) const {
  return data[row][col];
}

Matrix::operator Eigen::MatrixXd() const {
  Eigen::MatrixXd res(row, col);
  for(int i = 0; i < row; ++i) {
    for(int j = 0; j < col; ++j) {
      res(i, j) = data[i][j];
    }
  }
  return res;
}

ostream &operator<<(ostream &os, const Matrix &m) {
  os << fixed << setprecision(5);

  for(int i = 0; i < m.row; ++i) {
    for(int j = 0; j < m.col; ++j) {
      double val = m.data[i][j];
      if (abs(val) < 1e-5) val = 0.0;
      os << setw(12) << val << " ";
    }
    os << endl;
  }
```

```
  os.unsetf(ios_base::fixed);
  return os;
}

istream &operator>>(istream &is, Matrix &m) {
  for(int i = 0; i < m.row; ++i) {
    for(int j = 0; j < m.col; ++j) {
      is >> m.data[i][j];
    }
  }
  return is;
}

Matrix operator*(const Matrix &a, const Matrix &b) {
  if(a.col != b.row)
    throw invalid_argument("Kich thuoc 2 ma tran khi nhan khong hop le");
  Matrix res(a.row, b.col);
  for(int i = 0; i < a.row; ++i) {
    for(int j = 0; j < b.col; ++j) {
      double sum = 0;
      for(int k = 0; k < b.row; ++k) {
        sum += a.data[i][k] * b.data[k][j];
      }
      res.data[i][j] = sum;
    }
  }
  return res;
}

Matrix operator/(const Matrix &m, double k) {
  Matrix res(m.Row(), m.Col());
  for(int i = 0; i < m.Row(); ++i) {
    for(int j = 0; j < m.Col(); ++j) {
      res(i, j) = m(i, j) / k;
    }
  }
  return res;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới biết rằng $A = \begin{bmatrix} 6 & 2 & 3 \\ 6 & 2 & 5 \\ 6 & 2 & 7 \end{bmatrix}$, sai số $\varepsilon = 10^{-5}$.

```
Nhap kich thuoc ma tran: 3 3
Nhap ma tran A:
6 2 3
6 2 5
6 2 7
U:
     0.48228     -0.77507      0.40825
     0.57262     -0.07376     -0.81650
     0.66296      0.62755      0.40825

Sigma:
    14.07677      0.00000      0.00000
     0.00000      2.20106      0.00000
     0.00000      0.00000      0.00000

VT:
     0.73221      0.24407      0.63585
    -0.60322     -0.20107      0.77182
    -0.31623      0.94868      0.00000

U * Sigma * VT:
     6.00000      2.00000      3.00000
     6.00000      2.00000      5.00000
     6.00000      2.00000      7.00000
```

Câu 3 (*3.0 điểm, đáp ứng CLO1,CLO2, CLO3, CL04*)

Cho 20 điểm trong không gian Oxy như sau:
(1,1),(4,0),(7,1),(10,4),(9,8),(6,11),(2,10),(0,7),(0,3),(2,1),(3,3),(5,2),(7,4),(8,6),(7,8),(5,9),(3,8),(2,6),(4,5),(6,6)

a) (0.5 điểm) (0.5 điểm) Mô tả thuật toán xác định bao lồi của tập điểm trên (dạng sơ đồ khối hoặc mã giả)

Trả lời: dán sơ đồ khối hoặc mã giả:

Thuật toán:
Input: Tập hợp các điểm trên mặt phẳng.
Output: Danh sách các đỉnh của đa giác bao lồi theo chiều kim đồng hồ.
Các bước thực hiện:
Bước 1: Sắp xếp điểm

- Sắp xếp toàn bộ các điểm tăng dần theo hoành độ (x). Nếu hai điểm có cùng hoành độ, ưu tiên sắp xếp tăng dần theo tung độ (y).
- Kết quả ta được một danh sách đã sắp xếp: p1, p2, ..., pn. (p1 là điểm xa nhất bên trái, pn là điểm xa nhất bên phải).

Bước 2: Xác định Bao trên (Upper Hull - L_{upper})

- Khởi tạo danh sách L_{upper} và thêm hai điểm đầu tiên p1, p2 vào.
- Duyệt các điểm từ p3 đến pn:
    - Thêm điểm pi vào L{upper}.

- Kiểm tra: Nếu trong L{upper} có từ 3 điểm trở lên và 3 điểm cuối cùng không tạo thành một rẽ phải (tức là thẳng hàng hoặc rẽ trái, tương đương giá trị Định thức DET > 0), ta tiến hành xóa điểm ở giữa của 3 điểm đó.
- Lặp lại việc kiểm tra và xóa này cho đến khi 3 điểm cuối cùng tạo thành một rẽ phải hợp lệ.

Bước 3: Xác định Bao dưới (Lower Hull - L_{lower})

- Khởi tạo danh sách L{lower} và thêm hai điểm cuối cùng pn, p{n-1} vào.
- Duyệt ngược các điểm từ p_{n-2} lùi về p1:
  - Thêm điểm p_i vào L_{lower}.
  - Kiểm tra tương tự bước 2: Nếu 3 điểm cuối cùng trong L_{lower} không tạo thành rẽ phải (DET > 0), ta xóa điểm ở giữa. Lặp lại việc này đến khi có rẽ phải.

Bước 4: Hợp nhất Bao lồi

- Vì điểm p_1 và p_n xuất hiện ở cả hai tập L_{upper} và L_{lower}, ta cần xóa điểm đầu và điểm cuối của tập L_{lower} để tránh bị trùng lặp.
- Nối L_{upper} và L_{lower} lại với nhau. Ta thu được danh sách các điểm tạo thành bao lồi hoàn chỉnh.

Mã giả:

Input: Tập hợp P gồm n điểm (x, y) trên mặt phẳng.
Output: Danh sách L chứa các đỉnh của bao lồi theo chiều kim đồng hồ.

```
Hàm DET(A, B, C):
   Trả về (B.x - A.x) * (C.y - A.y) - (B.y - A.y) * (C.x - A.x)
   // DET <= 0: Rẽ phải (Hợp lệ)
   // DET > 0: Rẽ trái hoặc thẳng hàng (Không hợp lệ, cần xóa)
Bắt đầu:
   1. Sắp xếp mảng P tăng dần theo tọa độ x (nếu x bằng nhau thì ưu tiên y tăng dần).
   2. Khởi tạo danh sách Lupper (Bao trên) rỗng.
   3. Khởi tạo danh sách Llower (Bao dưới) rỗng.
     // Xây dựng Bao trên
   4. For i từ 1 đến n:
   5.     Thêm P[i] vào Lupper
   6.     While (kích thước Lupper >= 3) VÀ (DET(3 điểm cuối của Lupper) > 0):
   7.         Xóa điểm áp chót (điểm giữa của 3 điểm cuối) khỏi Lupper
     // Xây dựng Bao dưới
   8. For i từ n xuống 1:
   9.     Thêm P[i] vào Llower
   10.    While (kích thước Llower >= 3) VÀ (DET(3 điểm cuối của Llower) > 0):
   11.        Xóa điểm áp chót khỏi Llower
     // Hợp nhất
   12. Xóa điểm đầu và điểm cuối của Llower (để tránh trùng với Lupper).
   13. L = Lupper + Llower
   14. Return L
```

b) *(1.0 điểm)* Viết chương trình tìm bao lồi, sau đó tính cạnh nhỏ nhất của đa giác lồi tìm được.

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <iomanip>

using namespace std;

struct Point {
    long long x, y;
    bool operator<(const Point& p) const {
        return x < p.x || (x == p.x && y < p.y);
    }
};

long long crossProduct(Point O, Point A, Point B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}

vector<Point> convexHull(vector<Point>& A) {
    int n = A.size(), k = 0;
    if (n <= 2) return A;

    vector<Point> ans(2 * n);
    sort(A.begin(), A.end());

    for (int i = 0; i < n; ++i) {
        while (k >= 2 && crossProduct(ans[k - 2], ans[k - 1], A[i]) >= 0) k--;
        ans[k++] = A[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; --i) {
        while (k >= t && crossProduct(ans[k - 2], ans[k - 1], A[i]) >= 0) k--;
        ans[k++] = A[i];
    }

    ans.resize(k - 1);
    return ans;
}

double minEdge(const vector<Point>& hull) {
    int n = hull.size();
    if (n < 2) return 0.0;

    double min_dist = 1e18;

    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        double dist = sqrt((hull[i].x - hull[j].x) * (hull[i].x - hull[j].x) +
```

```
                    (hull[i].y - hull[j].y) * (hull[i].y - hull[j].y));
        if (dist < min_dist) {
            min_dist = dist;
        }
    }
    return min_dist;
}

int main() {
    int n;
    cout << "Nhap so diem n = ";
    if (!(cin >> n)) return 0;

    vector<Point> points(n);
    for (int i = 0; i < n; i++) {
        cout << "Nhap x" << i + 1 << " = ";
        cin >> points[i].x;
        cout << "Nhap y" << i + 1 << " = ";
        cin >> points[i].y;
    }

    vector<Point> hull = convexHull(points);
    cout << "Cac dinh cua Bao loi (Cung chieu kim dong ho):\n";
    for (const auto& p : hull) {
        cout << "(" << p.x << ", " << p.y << ")\n";
    }
    cout << "Canh nho nhat cua bao loi: " << fixed << setprecision(4) << minEdge(hull) <<
"\n";

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Cac dinh cua Bao loi (Cung chieu kim dong ho):
(0, 3)
(0, 7)
(2, 10)
(6, 11)
(9, 8)
(10, 4)
(7, 1)
(4, 0)
(1, 1)
Canh nho nhat cua bao loi: 2.2361
```

c) *(1.5 điểm)* Xác định chu vi và diện tích của đa giác bao lồi vừa tìm được.

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <iomanip>

using namespace std;

struct Point {
    long long x, y;

    bool operator<(const Point& p) const {
        return x < p.x || (x == p.x && y < p.y);
    }
};

long long crossProduct(Point O, Point A, Point B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}

vector<Point> convexHull(vector<Point>& A) {
    int n = A.size();
    int k = 0;

    if (n <= 2) return A;

    vector<Point> ans(2 * n);

    sort(A.begin(), A.end());

    // Xay nua duoi
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && crossProduct(ans[k - 2], ans[k - 1], A[i]) >= 0) {
            k--;
        }
        ans[k++] = A[i];
    }

    // Xay nua tren
    for (int i = n - 2, t = k + 1; i >= 0; --i) {
        while (k >= t && crossProduct(ans[k - 2], ans[k - 1], A[i]) >= 0) {
            k--;
        }
        ans[k++] = A[i];
    }
```

```
    ans.resize(k - 1);
    return ans;
}

// Tinh khoang cach giua 2 diem
double distancePoint(Point A, Point B) {
    double dx = A.x - B.x;
    double dy = A.y - B.y;
    return sqrt(dx * dx + dy * dy);
}

// Tinh chu vi da giac bao loi
double perimeter(const vector<Point>& hull) {
    int n = hull.size();

    if (n < 2) return 0.0;

    double p = 0.0;

    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        p += distancePoint(hull[i], hull[j]);
    }

    return p;
}

// Tinh dien tich da giac bao loi bang cong thuc Shoelace
double area(const vector<Point>& hull) {
    int n = hull.size();

    if (n < 3) return 0.0;

    long double s = 0.0;

    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        s += (long double)hull[i].x * hull[j].y - (long double)hull[j].x * hull[i].y;
    }

    return fabs(s) / 2.0;
}

int main() {
    int n;

    cout << "Nhap so diem n = ";
    cin >> n;
```

```
    vector<Point> points(n);

    for (int i = 0; i < n; i++) {
        cout << "Nhap x" << i + 1 << " = ";
        cin >> points[i].x;

        cout << "Nhap y" << i + 1 << " = ";
        cin >> points[i].y;
    }

    vector<Point> hull = convexHull(points);

    cout << "\nCac dinh cua bao loi la:\n";
    for (const auto& p : hull) {
        cout << "(" << p.x << ", " << p.y << ")\n";
    }

    cout << fixed << setprecision(4);

    cout << "\nChu vi cua da giac bao loi la: " << perimeter(hull) << endl;
    cout << "Dien tich cua da giac bao loi la: " << area(hull) << endl;

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Cac dinh cua bao loi la:
(0, 3)
(0, 7)
(2, 10)
(6, 11)
(9, 8)
(10, 4)
(7, 1)
(4, 0)
(1, 1)

Chu vi cua da giac bao loi la: 32.8977
Dien tich cua da giac bao loi la: 81.0000
```

Câu 4 (*2.0 điểm, đáp ứng CLO1,CLO2, CLO3, CL04*)

Cho hàm số $f(x,y) = (x^2 + y - 12)^2 + (x + y^2 - 9)^2$.

a) *(0.5 điểm) Trình bày thuật toán tối ưu hàm số đã cho* sử dụng phương pháp *gradient descent với momentum*, biết rằng tham số học (learning rate) γ, hệ số động lượng là α.

Trả lời: dán sơ đồ khối hoặc mã giả:

Thuật toán Gradient Descent with Momentum cho hàm f(x, y)

Input:

Điểm khởi tạo (x0, y0)

Tham số học gamma

Hệ số động lượng alpha

Số bước lặp tối đa N

Sai số epsilon

Output:

Điểm (x, y) làm f(x, y) nhỏ nhất gần đúng

Giá trị nhỏ nhất gần đúng f(x, y)

Bước 1:

Khởi tạo:

x = x0

y = y0

vx = 0

vy = 0

Bước 2:

Lặp i từ 1 đến N:

Tính đạo hàm riêng:

gx = ∂f/∂x

gy = ∂f/∂y

Nếu sqrt(gx^2 + gy^2) < epsilon:

Dừng thuật toán

Cập nhật động lượng:

vx = alpha * vx - gamma * gx

vy = alpha * vy - gamma * gy

Cập nhật biến:

x = x + vx

y = y + vy

Bước 3:

Trả về x, y và f(x, y)

Trong đó:

gamma là learning rate

alpha là momentum coefficient

b) *(1.5 điểm)* Viết chương trình (có dùng hàm) tính giá trị bé nhất của f(x) sử dụng phương pháp *gradient descent với momentum* với số bước lặp N và sai số ε.

Trả lời: **Dán code vào bên dưới:**

```
#include <iostream>
#include <cmath>
#include <iomanip>

using namespace std;

const double INF_LIMIT = 1e100;

double f(double x, double y) {
    double u = x * x + y - 12;
    double v = x + y * y - 9;

    return u * u + v * v;
}

double dfdx(double x, double y) {
    double u = x * x + y - 12;
    double v = x + y * y - 9;

    return 4 * x * u + 2 * v;
}

double dfdy(double x, double y) {
    double u = x * x + y - 12;
    double v = x + y * y - 9;

    return 2 * u + 4 * y * v;
}

bool isValid(double value) {
    return isfinite(value) && fabs(value) < INF_LIMIT;
}

void gradientDescentMomentum(
```

```
    double x,
    double y,
    double gamma,
    double alpha,
    int N,
    double epsilon
) {
    double vx = 0.0;
    double vy = 0.0;

    int iter = 0;
    bool converged = false;

    for (int i = 1; i <= N; i++) {
        iter = i;

        double gx = dfdx(x, y);
        double gy = dfdy(x, y);

        if (!isValid(gx) || !isValid(gy)) {
            cout << "Thuat toan bi tran so tai buoc lap: " << i << endl;
            break;
        }

        double vx_new = alpha * vx - gamma * gx;
        double vy_new = alpha * vy - gamma * gy;

        double x_new = x + vx_new;
        double y_new = y + vy_new;

        if (!isValid(x_new) || !isValid(y_new) || !isValid(f(x_new, y_new))) {
            cout << "Thuat toan bi phan ky tai buoc lap: " << i << endl;
            break;
        }

        double step = sqrt((x_new - x) * (x_new - x) + (y_new - y) * (y_new - y));
        double gradNorm = sqrt(dfdx(x_new, y_new) * dfdx(x_new, y_new)
                        + dfdy(x_new, y_new) * dfdy(x_new, y_new));

        if (step < epsilon && gradNorm < epsilon) {
            x = x_new;
            y = y_new;
            converged = true;
            break;
        }

        x = x_new;
        y = y_new;
```

```
        vx = vx_new;
        vy = vy_new;
    }

    cout << fixed << setprecision(10);

    if (converged) {
        cout << "Thuat toan hoi tu tai buoc lap: " << iter << endl;
    } else {
        cout << "Dat toi so buoc lap toi da hoac chua hoi tu.\n";
    }

    cout << "Gia tri x gan dung = " << x << endl;
    cout << "Gia tri y gan dung = " << y << endl;
    cout << "Gia tri nho nhat gan dung f(x, y) = " << f(x, y) << endl;
    cout << "Dao ham rieng theo x = " << dfdx(x, y) << endl;
    cout << "Dao ham rieng theo y = " << dfdy(x, y) << endl;
}

int main() {
    double x0, y0;
    double gamma, alpha, epsilon;
    int N;
    cout << "Nhap x0 = ";
    cin >> x0;

    cout << "Nhap y0 = ";
    cin >> y0;

    cout << "Nhap learning rate gamma = ";
    cin >> gamma;

    cout << "Nhap he so momentum alpha = ";
    cin >> alpha;

    cout << "Nhap so buoc lap N = ";
    cin >> N;

    cout << "Nhap sai so epsilon = ";
    cin >> epsilon;

    gradientDescentMomentum(x0, y0, gamma, alpha, N, epsilon);

    return 0;
}
```

Trả lời: **Dán kết quả thực thi** với điểm khởi $x=1, y=1,$ tham số học học (*learning rate*) $\gamma=0.001$, hệ số động lượng (*momentum coefficient*) là $\alpha=0.1,$ số bước lặp $N \geq 1000$ và sai số $\varepsilon=10^{-6}$:

```
Thuat toan hoi tu tai buoc lap: 357
Gia tri x gan dung = 3.0934880427
Gia tri y gan dung = 2.4303316762
Gia tri nho nhat gan dung f(x, y) = 0.0000000000
Dao ham rieng theo x = -0.0000004605
Dao ham rieng theo y = 0.0000008580
```

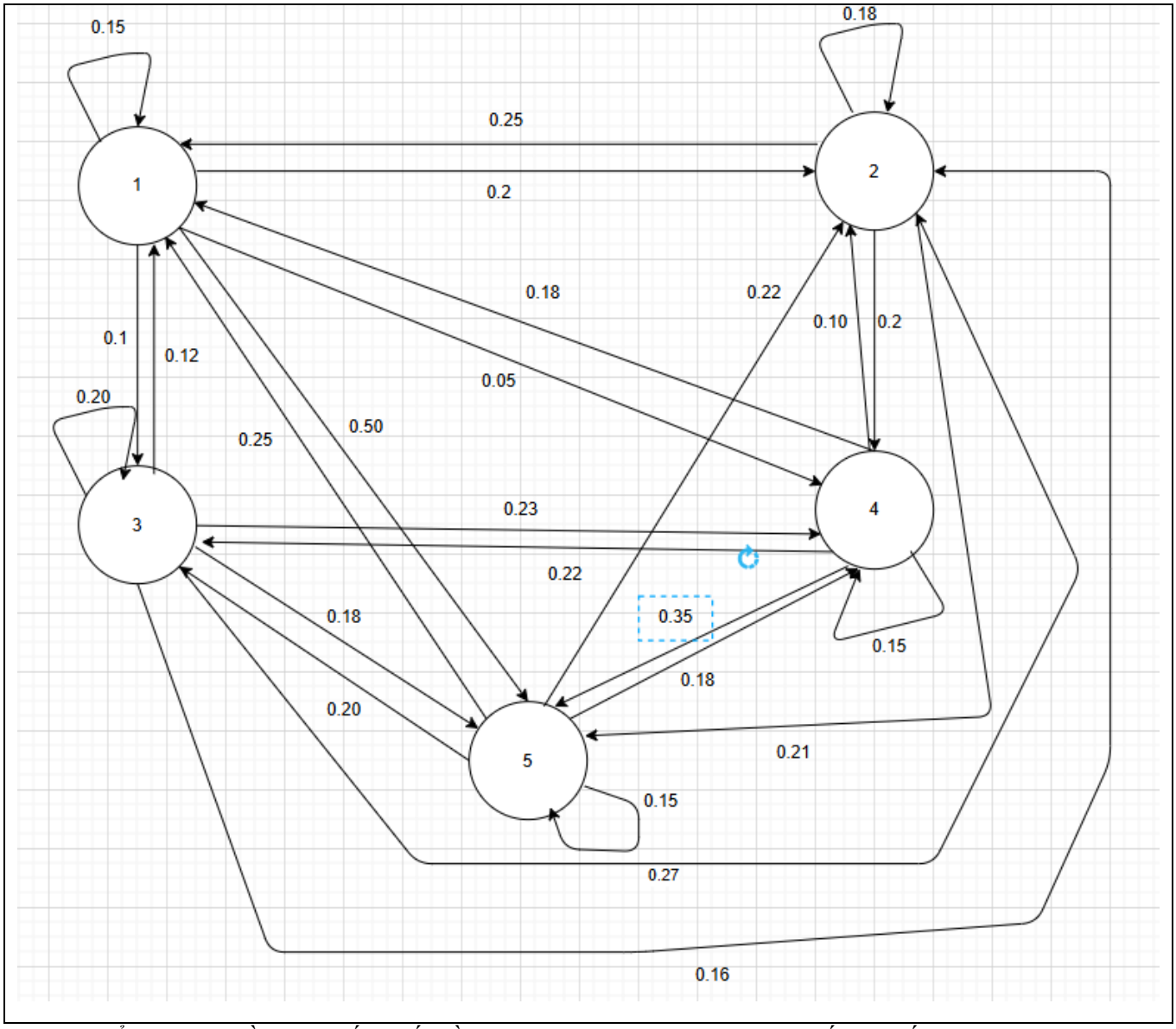
Câu 5 (*1.0 điểm, đáp ứng CLO1,CLO2, CL04*)

Một hệ thống có chế độ làm việc ở mỗi giai đoạn vận hành chỉ với các trạng thái từ 1 đến 5. Chế độ làm việc của hệ thống này được mô tả bằng ma trận chuyển như sau:

$$P=\begin{bmatrix} 0.15 & 0.20 & 0.10 & 0.05 & 0.50 \\ 0.25 & 0.18 & 0.16 & 0.20 & 0.21 \\ 0.12 & 0.27 & 0.20 & 0.23 & 0.18 \\ 0.18 & 0.10 & 0.22 & 0.15 & 0.35 \\ 0.25 & 0.22 & 0.20 & 0.18 & 0.15 \end{bmatrix}$$

a) (*0.5 điểm*) Vẽ đồ thị biểu diễn chuỗi Markov tương ứng đã cho

Trả lời: Dán kết quả vào bên dưới



b) (*1.0 điểm*) Giả sử rằng hệ thống bắt đầu học ở trạng thái 3. Tính xác xuất hệ thống làm việc ở trạng thái 5 *sau 3 bước thời gian vận hành,* và trạng thái 4 *sau 5 bước thời gian vận hành*

Trả lời: Dán kết quả tính toán vào bên dưới:

Ta có công thức phân phối xác suất sau k bước:

$$\pi(k) = \pi(0) \times P^k$$

Do hệ thống bắt đầu ở trạng thái 3 nên vector phân phối ban đầu là:

$$\pi(0) = [0 \quad 0 \quad 1 \quad 0 \quad 0]$$

1) Xác suất hệ thống ở trạng thái 5 sau 3 bước thời gian vận hành:

```
P^3 = [
  0.193250  0.196050  0.173190  0.159615  0.277895
  0.197929  0.198742  0.176370  0.163107  0.263852
  0.194673  0.196856  0.176300  0.162220  0.269951
  0.195623  0.197164  0.175796  0.163128  0.268289
  0.198331  0.198198  0.176706  0.163455  0.263310
]
```

Suy ra:

```
π(3) = π(0) × P^3 = [0.194673  0.196856  0.176300  0.162220  0.269951]
```

Vậy xác suất hệ thống làm việc ở trạng thái 5 sau 3 bước là 0.269951, tương đương 26.9951%.

2) Xác suất hệ thống ở trạng thái 4 sau 5 bước thời gian vận hành:

P^5 = [
  0.196031 0.197429 0.175643 0.162269 0.268628
  0.196244 0.197505 0.175764 0.162396 0.268091
  0.196126 0.197457 0.175722 0.162356 0.268338
  0.196159 0.197482 0.175735 0.162365 0.268259
  0.196261 0.197514 0.175774 0.162411 0.268041
]

Suy ra:

$\pi(5) = \pi(0) \times P^5 = [0.196126 \;\; 0.197457 \;\; 0.175722 \;\; 0.162356 \;\; 0.268338]$

Vậy xác suất hệ thống làm việc ở trạng thái 4 sau 5 bước là 0.162356, tương đương 16.2356%.

Kết luận:

- Sau 3 bước, xác suất hệ ở trạng thái 5 là 0.269951 (26.9951%).
- Sau 5 bước, xác suất hệ ở trạng thái 4 là 0.162356 (16.2356%).

Tổng cộng có: *05 câu*

Đà Nẵng, ngày…..tháng…..năm 2026

TRƯỞNG BỘ MÔN